

Paradigmas e Linguagens de Programação

Aula 11 Haskell

- **Histórico**
- **Introdução**
 - **Características**
- **Material de Haskell**

Histórico



So this is Haskell...



Haskell Brooks Curry

Matemático homenageado
pelos criadores da linguagem
Haskell

- Criada em ~1987, vários autores
- Linguagem **puramente funcional**
- Fortemente tipada, com inferência de tipos
- Suporta listas nativamente
- Implementação: compilador - interpretador



Origem

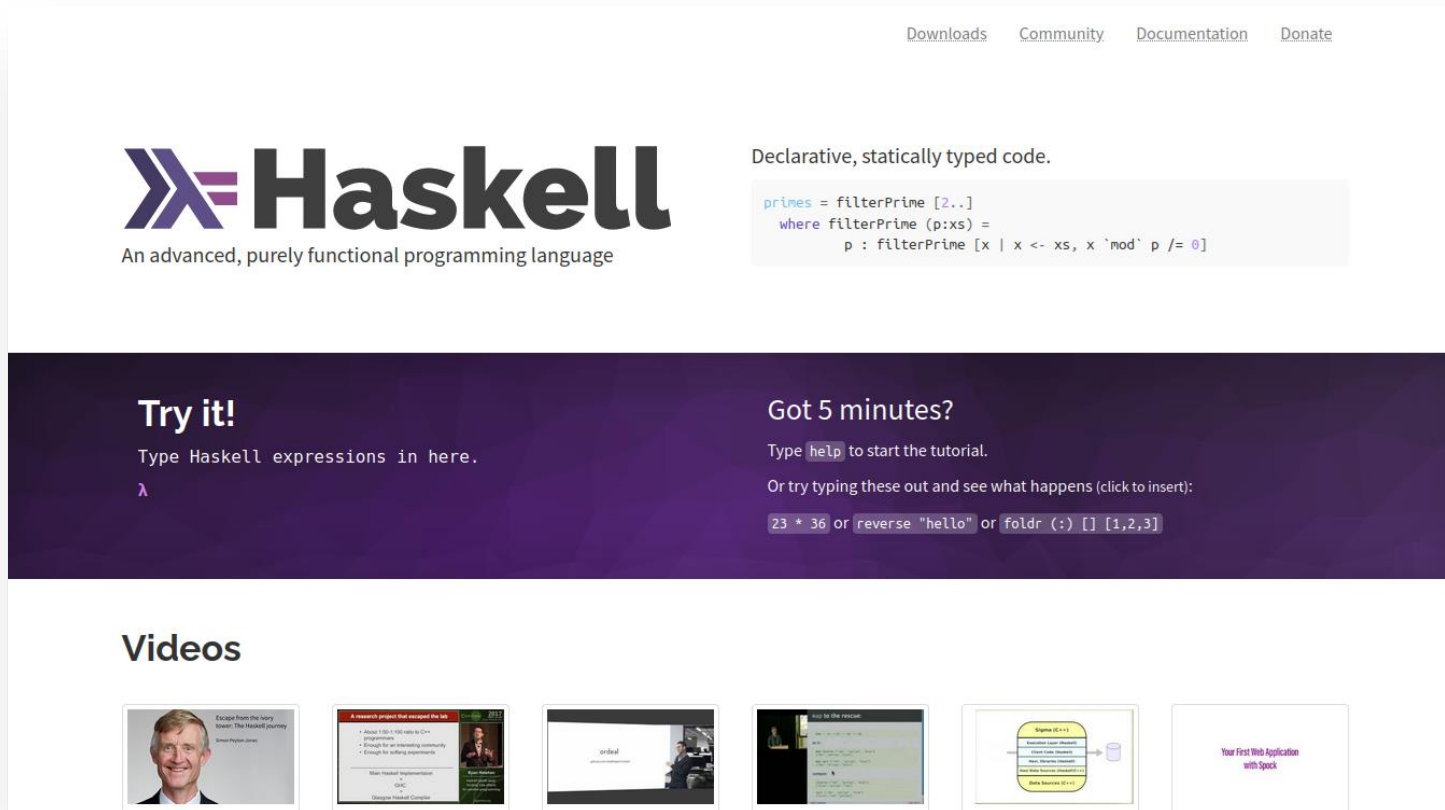
- 1987: Em uma **conferência realizada em Amsterdã**, a comunidade de programação funcional decidiu criar uma **linguagem puramente funcional** padronizada, designando um comitê para o desenvolvimento desse projeto.





Implementações Haskell

<http://haskell.org>



The screenshot shows the Haskell.org homepage. At the top, there are links for Downloads, Community, Documentation, and Donate. The main header features the Haskell logo (a stylized lambda symbol) and the text "Haskell" in a large, bold font, followed by the tagline "An advanced, purely functional programming language". To the right of the logo, it says "Declarative, statically typed code." and provides a code snippet for filtering primes. Below this, there are two sections: "Try it!" with a text input field and a "Got 5 minutes?" section with a tutorial link and sample code. At the bottom, there is a "Videos" section with a row of video thumbnails.

Downloads Community Documentation Donate

Haskell

An advanced, purely functional programming language

Declarative, statically typed code.

```
primes = filterPrime [2..]
where filterPrime (p:xs) =
  p : filterPrime [x | x <- xs, x `mod` p /= 0]
```

Try it!

Type Haskell expressions in here.

λ

Got 5 minutes?

Type `help` to start the tutorial.

Or try typing these out and see what happens (click to insert):

`23 * 36` or `reverse "hello"` or `foldr (:) [] [1,2,3]`

Videos

- Escape from the ivory tower: The Haskell journey
- A research project that changed the lab
- What Haskell is not
- What Haskell is
- What Haskell is not
- What Haskell is
- Your First Web Application with Spock



Origem

- 1987: Em uma **conferência realizada em Amsterdã**, a comunidade de programação funcional decidiu criar uma **linguagem puramente funcional** padronizada, designando um comitê para o desenvolvimento desse projeto.



Introdução



Programação Funcional



Linguagens funcionais **operam apenas sobre funções**, as quais **recebem listas de valores e retornam um valor**. O objetivo da programação funcional é **definir uma função que retorne um valor como a resposta do problema**.

Um **programa funcional** é uma chamada de função que **normalmente chama outras funções para gerar um valor de retorno**. As principais operações nesse tipo de programação **são a composição de funções e a chamada recursiva de funções**.

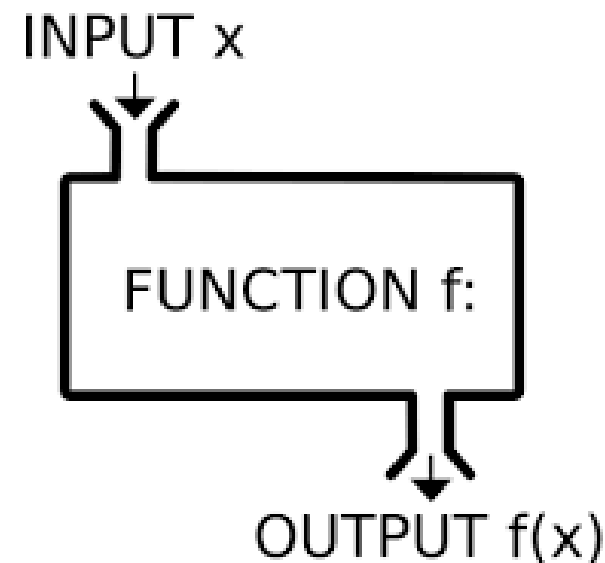
Outra característica importante é que funções são valores de primeira classe que podem ser passados para outras funções.



Programação Funcional

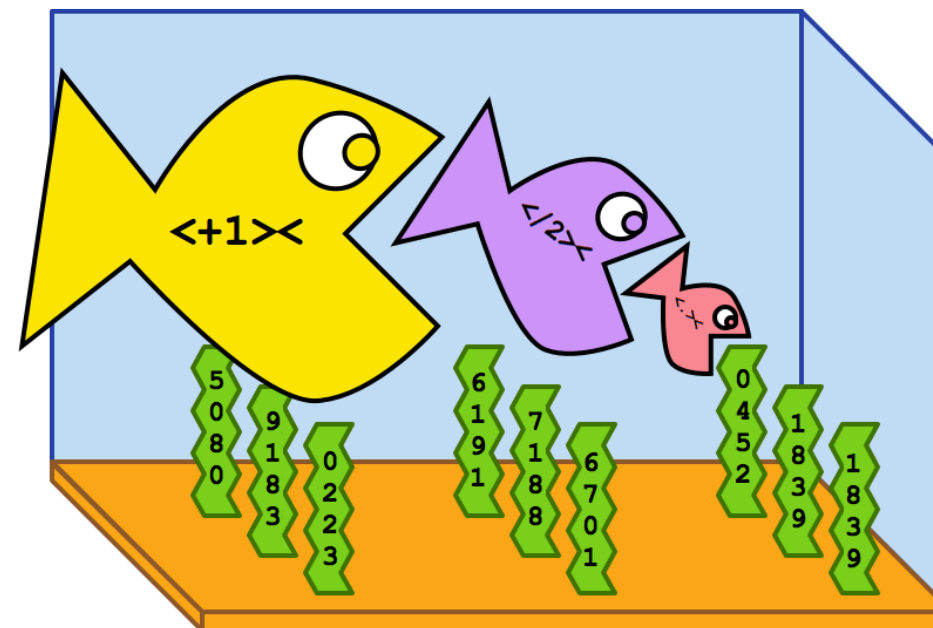
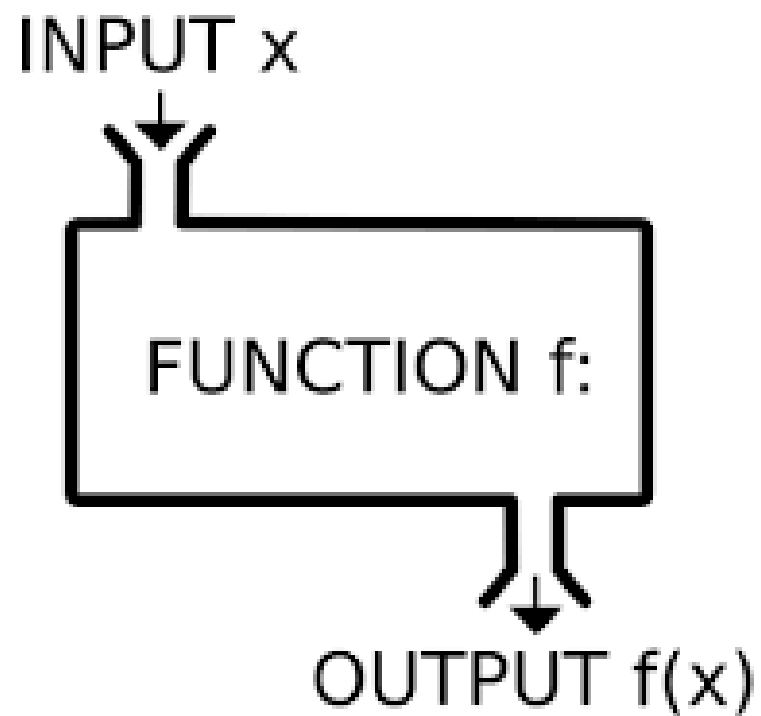
De maneira geral, pode-se pensar na **programação funcional** simplesmente como **avaliação de expressões**, cujo fluxo é:

- O programador define uma função para resolver um problema e a passa para o computador;
- **Uma função pode envolver várias outras funções em sua definição, inclusive si própria;**





Aplicando funções





Haskell



- **Haskell** – é uma linguagem de programação funcional, de propósito geral e de computação retardada(Lazy evaluation).
- **Computação retardada** – os programas em haskell são executados usando uma técnica chamada **avaliação preguiçosa** , que se baseia na ideia de que **nenhum calculo deve ser realizado até que o seu resultado seja realmente necessário.**

Características



Tipagem

Haskell é uma LP puramente funcional;

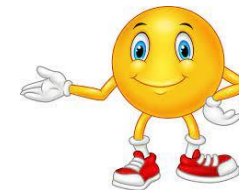


- Os tipos de dados são bem definidos, **sendo fortemente tipado**. Toda função, variável, constante tem um tipo de dado, que sempre pode ser determinado;
- Sistema de tipos estático;

Tipagem

Todos os tipos são conhecidos em tempo de compilação.

- **Existe inferências de tipos**. Não é preciso explicitar de qual tipo é um certo identificador.
- Haskell permite que **um mesmo identificador seja declarado em diferentes partes do programa, possivelmente representando diferentes entidades**.
- As conversões de dados **são explícitas**.





Operadores Básicos

>	Maior	+	Soma
>=	Maior ou igual	-	Subtração
==	Igual	*	Multiplicação
/=	Diferente	^	Potência
<	Menor	div	Divisão inteira
<=	Menor ou igual	mod	Resto da divisão
&&	e	abs	valor absoluto de um inteiro
	ou	negate	troca o sinal do valor
not	Negação	:	Composição de lista
++	Concatenação		



Palavras Reservadas

case	class	data	deriving	do
else	if	import	in	infix
infixl	infixr	instance	let	of
module	newtype	then	type	where



Ambiente para executar Haskell



- https://www.onlinegdb.com/online_haskell_compiler

Material de Haskell

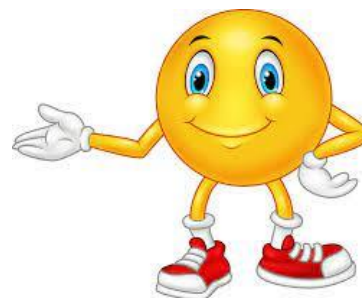


Material de Haskell

<http://haskell.org>

<https://learnyouahaskell.com/>

<https://wiki.haskell.org/>



An advanced, purely functional programming language



Referências Bibliográficas

